

SESSION 10

Strings in Python

Strings: Quick Recap and What's Next

We briefly encountered strings in an earlier session. Let's quickly recap their creation and basic usage before delving deeper into their powerful features.

```
# Creating strings — single, double, or triple quotes
name  = "Rahul"
city  = 'Delhi'
message = """This is a
multi-line string"""

# Printing strings
print(name)
print(f"Hello, {name} from {city}") # f-string

# Strings can contain any character
address = "Flat No. 404, Pune - 411001"
```

In this session, we'll explore strings much more comprehensively — how to access individual characters, slice specific portions, search for patterns, modify case, split them into pieces, and utilise the numerous built-in methods Python offers for text manipulation.

Strings are fundamental in real-world programming, appearing in usernames, emails, file content, API responses, and user input. Mastering them is essential for any Python developer.

Strings are Immutable

In Python, strings are immutable. This means that once a string object is created, you cannot change its individual characters directly or modify it in place. Any operation that appears to "change" a string actually results in the creation of a brand new string object.

```
name = "Rahul"

# This will CRASH with a TypeError!
# name[0] = "K"  # TypeError: 'str' object does not support item assignment
```

Attempting to assign a new character to an index within an existing string will raise a `TypeError`, as strings do not support item assignment. To achieve a modification, you must construct a new string from parts of the old one or by concatenation.

```
# The correct way: create a new string
name = "K" + name[1:]
print(name)  # Output: Kahul
```

This principle also applies to operations like string concatenation. Even when you add two strings together, Python creates a completely new string to store the result.

```
greeting = "Hello"
greeting = greeting + " World" # A new string "Hello World" is created
print(greeting)  # Output: Hello World
```

 **Key Takeaway:** Whenever you perform an operation that seems to alter a string, remember that Python is creating a new string behind the scenes. The original string remains unchanged in memory, and the variable is simply reassigned to point to the new string.

Indexing Strings

Every character in a string has a specific position, known as an index. This allows you to access individual characters or parts of a string. Python uses two types of indexing:

Positive Indexing (Left to Right)

Indexes start from 0 for the first character and increase by one for each subsequent character towards the right.

```
name = "Python"
#    0 1 2 3 4 5 (Positive Indexes)

print(name[0])  # Output: P
print(name[3])  # Output: h
print(name[5])  # Output: n
```

Negative Indexing (Right to Left)

Indexes start from -1 for the last character and decrease as you move towards the left.

```
name = "Python"
#   -6 -5 -4 -3 -2 -1 (Negative Indexes)

print(name[-1])  # Output: n
print(name[-3])  # Output: h
print(name[-6])  # Output: P
```

This allows for convenient access to characters from either end of the string. For example, to get the first and last character of a word:

```
word = input("Enter a word: ")
print(f"First character: {word[0]}")
print(f"Last character: {word[-1]}")
```

⚠ **Watch Out:** Accessing an index that doesn't exist will raise an `IndexError`. If a string has 6 characters, valid positive indices are 0 to 5, and valid negative indices are -1 to -6.

Slicing Strings

Slicing is a powerful technique that allows you to extract a specific portion or "slice" from a string. It works similarly to how you slice lists in Python, using a `start`, `stop`, and optional `step` index.

```
text = "Programming"
#    0 1 2 3 4 5 6 7 8 9 10

# Basic Slicing
print(text[0:4])    # Output: Prog
print(text[3:8])    # Output: gramm
print(text[:5])     # Output: Progr   (start defaults to 0)
print(text[5:])     # Output: amming   (stop defaults to end)
print(text[::-2])   # Output: Pormig   (every 2nd character)
print(text[::-1])   # Output: gnimmargorP (reversed)
```

This flexible syntax allows for a wide range of substring extraction, from simple sections to more complex patterns or even reversing the entire string.

Practical Applications

```
email = "rahul@gmail.com"
username = email[:email.index("@")] # Extracts "rahul"
domain = email[email.index("@")+1:] # Extracts "gmail.com"

print(f"Username: {username}")
print(f"Domain: {domain}")

# Reversing a string is a one-liner
word = "hello"
print(word[::-1]) # Output: olleh
```

 **Quick Tip:** `string[::-1]` is the cleanest one-liner to reverse a string in Python. It's a highly useful idiom that you'll encounter frequently in coding challenges and real-world scenarios.

Concatenation and Repetition

Strings can be combined (concatenated) using the `+` operator and repeated using the `*` operator, creating new string objects in the process.

Concatenation using `+`

The addition operator `+` joins two or more strings together to form a new, longer string.

```
first = "Rahul"
last  = "Sharma"

full_name = first + " " + last
print(full_name)  # Output: Rahul Sharma

# Building messages
greeting = "Hello, " + first + "!"
print(greeting)   # Output: Hello, Rahul!
```

Repetition using `*`

The multiplication operator `*` repeats a string a specified number of times.

```
print("Ha" * 3)      # Output: HaHaHa
print("-" * 30)      # Output: -----
print("Python " * 4) # Output: Python Python Python Python
```

Practical Applications

These operations are incredibly useful for formatting output, creating separators, or generating repetitive patterns.

```
# Drawing separators or patterns
print("=" * 40)
print("    Code and Debug    ")
print("=" * 40)

# A simple right-aligned name banner
name = "Rahul"
print(" " * (40 - len(name)) + name)
```

 **Watch Out:** The `+` operator requires both operands to be strings. Concatenating a string with a non-string type (e.g., `"Age: " + 21`) will raise a `TypeError`. Always convert numbers to strings using `str()` or use f-strings for mixed-type concatenation.

Looping Through a String

Iterating over characters in a string is a fundamental operation in Python, allowing you to access and process each character sequentially. This is useful for various tasks like searching, counting, or modifying strings.

Using a for loop

The simplest way to iterate directly over each character:

```
name = "Python"
for char in name:
    print(char)
# Output:
# P
# y
# t
# h
# o
# n
```

Using a while loop

For more control, you can use a while loop with an index:

```
name = "Python"
i = 0
while i < len(name):
    print(name[i])
    i += 1
```

Using enumerate()

When you need both the character and its index simultaneously:

```
name = "Python"
for index, char in enumerate(name, start=1):
    print(f"{index}: {char}")
# Output:
# 1: P
# 2: y
# 3: t
# 4: h
# 5: o
# 6: n
```

Practical Application: Count Vowels

Here's how you can count vowels in a sentence using a for loop:

```
sentence = "Welcome to Python programming"
vowels = "aeiouAEIOU"
count = 0
for char in sentence:
    if char in vowels:
        count += 1
print(f"Total vowels: {count}") # Output: Total vowels: 8
```

Membership Check

Python provides the `in` and `not in` operators to easily check if a character or substring exists within a string. This is extremely useful for searching and validation tasks.

```
sentence = "Welcome to Python programming"

# Check for single character
print("P" in sentence)    # Output: True
print("z" in sentence)    # Output: False

# Check for substring
print("Python" in sentence) # Output: True
print("Java" in sentence)  # Output: False

# Using 'not in'
print("Java" not in sentence) # Output: True
print("to" not in sentence)  # Output: False
```

Practical Use: Input Validation

Membership checks are often used to validate user input, ensuring it meets certain criteria.

```
email = input("Enter your email: ")

if "@" in email and "." in email:
    print("Looks like a valid email address.")
else:
    print("Invalid email format. Please include '@' and '.'")
```

 **Watch Out:** Membership checks in Python are **case-sensitive**. `"python"` in `"Python is fun"` will return `False`. To perform a case-insensitive check, convert both the string and the substring to the same case (e.g., lowercase) before comparison using methods like `.lower()` or `.upper()`.

Built-in Functions on Strings

Python offers several built-in functions that provide quick and efficient ways to perform common operations on strings, helping you extract information or modify their structure without complex code.



len(): Get Length

Returns the total number of characters in a string, including spaces and special characters. It's useful for checking string size or for loop conditions.

```
text = "Programming"  
print(len(text)) # Output: 11
```



min() and max(): Alphabetical Extremes

These functions return the alphabetically smallest (min()) and largest (max()) character in a string. This comparison is based on the character's Unicode value.

```
print(min("apple")) # Output: a  
print(max("apple")) # Output: p  
print(min("Banana")) # Output: B  
(Uppercase comes before  
lowercase)
```



sorted(): Sorted Characters

The sorted() function takes an iterable (like a string) and returns a new list containing all items from the string in ascending order. The original string remains unchanged.

```
print(sorted("python"))  
# Output: ['h', 'n', 'o', 'p', 't', 'y']
```

Note: min(), max(), and sorted() perform comparisons based on the character's Unicode values. This means uppercase letters come before lowercase letters (e.g., 'A' < 'a'), which can sometimes lead to unexpected sorting if not accounted for.

Practice Questions: Strings (Batch 1)

Test your understanding of Python strings with these challenges. Try to solve them using the concepts covered so far!

Q93. Take a name as input from the user. Print its first character, its last character, and the total length of the name.

Q95. Read a sentence from the user. Count and print the total number of vowels (a, e, i, o, u, case-insensitive) present in it, using a `for` loop. **(HW)**

Q94. Accept a string as input. Print its reverse using string slicing.

Q96. Given a string input, print every alternate character starting from the very first character, using string slicing. **(HW)**

String Methods: Case Conversion

Python provides several built-in string methods to easily change the case of characters within a string. It's crucial to remember that these methods return a **new string** with the modified case; the original string remains **unchanged** because strings in Python are immutable.

```
text = "Hello World From Python"

print(text.upper())    # Output: HELLO WORLD FROM PYTHON
print(text.lower())    # Output: hello world from python
print(text.title())    # Output: Hello World From Python (Title Case)
print(text.capitalize()) # Output: Hello world from python (only first char)
print(text.swapcase()) # Output: hELLO wORLD fROM pYTHON (flips each)

print(text)           # Output: Hello World From Python (original string unchanged)
```

- `upper()`: Converts all characters in the string to uppercase.
- `lower()`: Converts all characters in the string to lowercase.
- `title()`: Converts the first character of each word to uppercase and the remaining to lowercase.
- `capitalize()`: Converts only the first character of the entire string to uppercase and the rest to lowercase.
- `swapcase()`: Swaps the case of each character – uppercase becomes lowercase and vice versa.

Practical Applications

Case conversion methods are widely used for data normalization, user input processing, and display formatting.

Case-Insensitive Comparison

Ensure your program responds correctly regardless of how users type their input:

```
user_input = input("Type 'yes' or 'no': ") # User might type "Yes", "YES", "yEs"

if user_input.lower() == "yes":
    print("Confirmed.")
elif user_input.lower() == "no":
    print("Cancelled.")
else:
    print("Invalid input.")
```

 **Quick Tip:** When comparing user input to expected values, always convert both to the same case first. This avoids bugs caused by users typing in mixed case.

Formatting Names Properly

Easily format names or titles to have consistent capitalization:

```
name = "rahul kumar"
print(name.title())    # Output: Rahul Kumar — perfect for displaying names
```

String Methods: Checking Content

Python provides handy string methods that return boolean values (True or False) based on the content and characteristics of the string. These are excellent for input validation and data cleaning.

isalpha(): All Letters?

Returns True if all characters in the string are alphabetic and the string is not empty; otherwise, returns False.

```
print("Hello".isalpha())    # Output: True
print("Hello123".isalpha()) # Output: False
print("Hello World".isalpha()) # Output: False (space breaks it)
```

isdigit(): All Digits?

Returns True if all characters in the string are digits and there is at least one character; otherwise, returns False.

```
print("12345".isdigit())    # Output: True
print("12.5".isdigit())     # Output: False
print("abc123".isdigit())   # Output: False
```

isalnum(): AlphaNumeric Only?

Returns True if all characters in the string are alphanumeric (letters or numbers) and there is at least one character; otherwise, returns False.

```
print("Hello123".isalnum()) # Output: True
print("Hello 123".isalnum()) # Output: False
```

isspace(): All Whitespace?

Returns True if all characters in the string are whitespace characters and there is at least one character; otherwise, returns False.

```
print(" ".isspace())        # Output: True
print("a ".isspace())       # Output: False
```

Checking for Prefixes and Suffixes

Beyond character types, you can also check if a string begins or ends with a specific sequence of characters using startswith() and endswith().

```
filename = "report_2024.pdf"

print(filename.startswith("report")) # Output: True
print(filename.endswith(".pdf"))     # Output: True
print(filename.endswith(".docx"))    # Output: False
```

Practical Use: Input Validation

These methods are vital for validating user input, ensuring your program receives data in the expected format before processing it, preventing errors and improving reliability.

```
age = input("Enter your age: ")

if age.isdigit():
    age = int(age)
    print(f"You are {age} years old.")
else:
    print("Please enter a valid number.")
```

 **Tip:** These "is" methods are often chained or combined with logical operators (and, or) to create more complex validation rules, ensuring data integrity in your applications.

String Methods: Searching and Replacing

Python offers a powerful set of built-in string methods to locate substrings, count their occurrences, and replace them. These methods are essential for text processing, data manipulation, and input validation in various applications.

```
sentence = "Python is great and Python is powerful"
```

count(): Substring Frequency

Returns the number of non-overlapping occurrences of a substring in the string. If the substring is not found, it returns 0.

```
print(sentence.count("Python")) # Output: 2
print(sentence.count("is"))    # Output: 2
print(sentence.count("Java"))  # Output: 0
```

find(): Locate First Occurrence

Returns the lowest index in the string where the substring is found. If the substring is not found, it returns -1. This method does not raise an error.

```
print(sentence.find("great")) # Output: 10
print(sentence.find("Java"))  # Output: -1
```

index(): Locate (with Error)

Similar to find(), but raises a ValueError if the substring is not found. Use this when you are certain the substring will be present.

```
print(sentence.index("great")) # Output: 10
# print(sentence.index("Java")) # ValueError: substring not found
```

replace(): Substitute Substrings

Returns a **new string** with all occurrences of an old substring replaced by a new one. An optional third argument specifies the maximum number of replacements.

```
print(sentence.replace("Python", "Java"))
# Output: Java is great and Java is powerful

# Replace only the first occurrence
print(sentence.replace("Python", "Java", 1))
# Output: Java is great and Python is powerful
```

Practical Use: Cleaning User Input

These methods are invaluable for sanitising user-provided data, removing unwanted characters, or standardising formats before processing.

```
phone = "+91-98765-43210"
```

```
# Remove dashes and country code by chaining replace()
clean = phone.replace("-", "").replace("+91", "")
print(clean) # Output: 9876543210
```

📌 **Quick Tip:** Use `find()` when you're uncertain if a substring exists, as it returns -1 gracefully. Reserve `index()` for situations where you expect the substring to be present and want an error if it's not.

String Methods: Splitting and Joining

Python's `split()` and `join()` methods are fundamental for manipulating text, allowing you to easily convert strings into lists of substrings and vice-versa. These are indispensable tools for parsing data and constructing formatted output.

`split()`: Break String into List

This method breaks a string into a list of substrings. By default, it splits the string by any whitespace. You can also specify a custom delimiter.

```
pythonsentence = "Python is fun and powerful"
```

```
# Split by space (default)
```

```
words = pythonsentence.split()
```

```
print(words)
```

```
# Output: ['Python', 'is', 'fun', 'and', 'powerful']
```

```
# Split by a specific character (e.g., comma in CSV)
```

```
csv = "apple,mango,banana,grapes"
```

```
fruits = csv.split(",")
```

```
print(fruits)
```

```
# Output: ['apple', 'mango', 'banana', 'grapes']
```

```
# Splitting an email address
```

```
email = "rahul@gmail.com"
```

```
parts = email.split("@")
```

```
print(parts)      # Output: ['rahul', 'gmail.com']
```

```
print(parts[0])   # Output: rahul
```

```
print(parts[1])   # Output: gmail.com
```

`join()`: Combine List into String

The `join()` method allows you to concatenate elements of an iterable (like a list) into a single string. The string on which `join()` is called acts as the separator between the elements.

```
words = ["Python", "is", "fun"]
```

```
# Join with space as separator
```

```
sentence = " ".join(words)
```

```
print(sentence)   # Output: Python is fun
```

```
# Join with hyphen as separator
```

```
result = "-".join(words)
```

```
print(result)     # Output: Python-is-fun
```

```
# Join with no separator
```

```
result = "".join(words)
```

```
print(result)     # Output: Pythonisfun
```

```
# Practical use: Formatting a shopping list
```

```
items = ["bread", "milk", "eggs", "butter"]
```

```
print("Shopping list: " + ", ".join(items))
```

```
# Output: Shopping list: bread, milk, eggs, butter
```

Quick Tip: `split()` and `join()` are often inverse operations. For example, `"-".join("a-b-c".split("-"))` gives you back the original string. These two methods are heavily used in real-world Python applications.

Practice Questions: Strings (Batch 2)

Challenge your understanding of Python string methods with these practical questions. Work through them to solidify your skills!

Q97: Email Validation

Take an email as input. Validate that it contains exactly one @ and at least one .. Print "Valid" or "Invalid".

Q98: Clean Phone Number

Take a phone number as input in the format +91-98765-43210. Remove all dashes and the country code. Print the cleaned 10-digit number.

Q99: Vowel-Starting Words

Take a sentence as input. Split it into words and print how many words start with a vowel.

Q100: Sorted Names

Take a list of names as input (comma separated). Split them, sort them alphabetically, and join them back with " | " as separator.

Q101: File Type Check

Take a filename as input (like report.pdf). Check if it ends with .pdf, .docx, or .txt and print the file type.

Q102: Longest Word

Take a sentence as input. Print the longest word in it.

String Methods: `strip()`, `lstrip()`, `rstrip()`

These methods are crucial for data cleaning, allowing you to remove unwanted whitespace or specified characters from the beginning, end, or both sides of a string. This ensures data consistency and prevents errors in comparisons.

`strip()`: Remove Leading/Trailing

Removes all whitespace characters (spaces, tabs, newlines) from **both ends** of the string. The original string remains unchanged.

```
pythontext = " Hello World "  
print(pythontext.strip()) # Output: "Hello World"
```

`lstrip()`: Remove Leading

Removes all whitespace characters from the **left (leading)** end of the string. Useful for left-justifying text or cleaning data entry.

```
pythontext = " Hello World "  
print(pythontext.lstrip()) # Output: "Hello World "
```

`rstrip()`: Remove Trailing

Removes all whitespace characters from the **right (trailing)** end of the string. Ideal for removing trailing newlines from file reads or right-justified text.

```
pythontext = " Hello World "  
print(pythontext.rstrip()) # Output: " Hello World"
```

Stripping Specific Characters

Beyond whitespace, these methods can also remove specific characters if provided as an argument. The argument is treated as a set of characters to be removed from the ends of the string.

```
url = "https://www.codeanddebug.com/"  
  
# Remove leading "https://" and trailing "/"  
clean_url = url.strip("https:/")  
print(clean_url) # Output: www.codeanddebug.com  
  
text = "###Important###"  
print(text.lstrip("#")) # Output: Important###  
print(text.rstrip("#")) # Output: ###Important  
print(text.strip("#")) # Output: Important
```

 **Quick Tip:** Always `strip()` user input before processing it. Users often add accidental spaces or newlines, which can cause subtle bugs in comparisons and lookups.

Practice Questions: Strings (Batch 3)

Continue to challenge your string manipulation skills with this new set of practical questions. Work through them to master these important Python methods!

Q103: Username Validation

Take a username as input. Strip any spaces from both sides and check if the cleaned name starts with a letter (not a digit). Print "Valid" or "Invalid".

Q104: Unique Words Count

Take a paragraph as input. Print the count of unique words in it (case insensitive).

Q105: Palindrome Check

Write a function `is_palindrome(text)` that returns `True` if the given string is a palindrome (ignoring case and spaces).

Q106: Word Lengths (Homework)

Take a sentence as input. Print each word's length next to it. Example: Python (6) is (2) great (5).

Q107: Reverse Word Order

Take a sentence as input. Reverse the order of words (not the characters in each word). Example: "Python is fun" → "fun is Python".

Q108: Capitalize Long Words (Homework)

Take a sentence as input. Capitalize only the first letter of each word that is longer than 3 characters. Keep the rest as is.

Q109: PAN Card Validation (Homework)

Take a string as input. Check if it is a valid Indian PAN card number. Format: 5 uppercase letters + 4 digits + 1 uppercase letter (Total 10 characters). Example: ABCDE1234F.

Q110: Most Common Word (Homework)

Write a function `most_common_word(text)` that returns the word that appears most frequently in a given paragraph. Ignore case. If multiple words have the same count, return any one.